Risk-Sensitive Distributional Offline RL for Cryptocurrency Trading

Aryan Jain
Cockrell School of Engineering
University of Texas at Austin
Austin, TX 78712
aryanj179@utexas.edu

Ronak Jain
Cockrell School of Engineering
University of Texas at Austin
Austin, TX 78712
ronakjain0919@utexas.edu

Abstract

We study offline reinforcement learning for systematic trading on Bitcoin using only past data, not real-time interaction. We build a full pipeline (daily BTC-USD data, engineered features, chronological splits, portfolio costs, Gymnasium-style `OfflineTradingEnv`) and an offline dataset from twelve behavior policies on the training years. Train, validation, and test periods correspond to different market regimes (bull, bear or sideways, and recovery bull), while the offline data are drawn from a long bull phase, so agents must generalize outside the training distribution. We compare a regularized Double DQN, discrete CQL (α=1), and a distributional conservative agent (DistCQL: quantile critics with mean and tail CQL penalties and ensemble pessimism) against buy-and-hold. We evaluate all methods across five random seeds and report statistical significance tests following Henderson et al. (2018). CQL is the most seed-robust model (mean test return $+162\%$). An ensemble averaging DistCQL (α=0.5) and CQL positions creates a consensus gate: across all 25 seed combinations, 96% produce positive returns and 76% beat buy-and-hold. The ensemble significantly outperforms buy-and-hold (one-sample t-test, p=0.002; mean $+154\%$ vs. $+98\%$); the seed-42 ensemble achieves $+238\%$ test return, Sharpe 1.09, and maximum drawdown -22%, while the best pair in the grid reaches $+288\%$. Two-sample t-tests across seeds show CQL beats DQN on both test returns (p=0.008) and Sharpe (p=0.042).

1 Introduction

Reinforcement learning (RL) is increasingly applied to cryptocurrency portfolio management. However, most prior work relies on online paradigms (e.g., PPO, SAC) requiring continuous, risky interaction with simulated or live, non-stationary markets (2). To avoid costly trial-and-error, recent efforts have explored offline (batch) RL, learning strictly from static historical datasets (3). Yet, standard offline methods often fail in financial domains because they blindly extrapolate Q-values for out-of-distribution (OOD) market states and optimize only for expected returns, ignoring severe downside risk.

To address these shortcomings, this project develops conservative and distributional offline RL on Bitcoin (BTC) using daily-frequency data and discrete actions (short, flat, long). We detail our pipeline, dataset construction, mean CQL (4), and a distributional variant (DistCQL), evaluated against buy-and-hold and a Double DQN baseline. Risk-sensitive control on top of the learned quantiles is left as a direct extension.

A regularized Double DQN fits validation well but fails out of sample, with no seed beating buy-and-hold. CQL (α=1) trades lower headline validation numbers for more honest value estimates and the strongest individual test performance (mean +162%), though its seed variance leaves it only marginally significant against buy-and-hold. DistCQL at α=1 collapses to passive holding; however,

reducing to α=0.5 can produce an active distributional trader, but with high seed sensitivity. An ensemble that averages DistCQL (α=0.5) and CQL positions creates a consensus gate that stabilizes returns across seed combinations. CQL significantly outperforms DQN on both test returns (p=0.008) and Sharpe (p=0.042), and the DistCQL+CQL ensemble significantly beats buy-and-hold (p=0.002). The ensemble is positive in 96% of seed combinations and beats buy-and-hold in 76%.

2 Related Work

Early and modern work applies RL to portfolio choice and execution, often under simplifying assumptions (e.g., simulated prices or low-frequency rebalancing) (2). Offline RL algorithms learn from a fixed dataset without collecting new transitions (3). Naive application of deep Q-learning (1) can suffer from overestimation and extrapolation error when the learned policy visits states not well covered by data. Conservative methods such as Conservative Q-Learning penalize Q-values for out-of-distribution actions to reduce over-optimism (4). These ideas are particularly relevant when behavior data come from simple heuristics rather than a near-optimal expert.

Instead of estimating only the mean return under each action, distributional approaches model the full return distribution (5) (e.g., via quantile regression (6)). In finance, tails and downside risk matter as much as the mean; distributional outputs enable risk-sensitive policies (e.g., lower quantiles or tail risk measures). Practical trading systems must model fees, slippage, and turnover. Our implementation incorporates proportional transaction costs in a portfolio simulator so that rewards reflect net rather than gross returns.

3 Data

Daily BTC-USD candles are pulled from the Coinbase Exchange API, normalized to UTC, forward-filled for gaps of up to two days, and stored as Parquet. The series is split chronologically into train, validation, and test periods (summarized in Table 1). The three splits span distinct regimes: training covers a strong bull phase, validation is a bear or sideways market, and test is a recovery bull. Because the offline dataset is built exclusively from training-period episodes, states in which long exposure paid off are overrepresented relative to the later splits, a distributional mismatch that any offline learner must contend with.

Table 1: Approximate regime statistics for BTC-USD (buy-and-hold on each split).

Split	Period	Cumulative return	Daily vol	Regime (informal)
Train	Jul 2017 to Dec 2021	+1048%	4.28%	Strong bull
Val	Jan 2022 to Dec 2023	−7.4%	2.90%	Bear or sideways
Test	Jan 2024 to Dec 2025	+97.9%	2.50%	Recovery bull

Features are engineered on the full aligned series before the chronological split, and standardization statistics are computed on the training period only; validation and test reuse those statistics unchanged to prevent leakage. The feature set captures price momentum and volatility across multiple horizons, including log returns over 1-, 5-, and 20-day periods, realized volatility over 5 and 20 days, and price-to-SMA ratios at 10, 20, and 50 days. It is further enriched with standard technical indicators such as RSI-14 and MACD (line, signal, and histogram), along with log-transformed volume, its first difference, and pivot-based distances to nearby supply and demand zones.

From the training split we generate transitions $(s_t, a_t, r_t, s_{t+1}, \text{done}_t)$ by rolling out twelve behavior policies: seven core strategies (buy-and-hold, random, trend-following, mean-reversion, MACD crossover, supply/demand zones, and a volatility regime filter), four ε-greedy wrappers ($\varepsilon = 0.15$) that inject exploration noise on top of the signal-based policies, and one weighted mixture that samples across all seven philosophies per step. This diversity ensures that every discrete action (short, flat, long) has meaningful coverage in the dataset, which is a prerequisite for CQL's conservative penalty to behave correctly. The resulting episodes concatenate into roughly 10^4 transitions, which are stored as an `.npz` archive and converted to `d3rlpy`'s `MDPDataset` at training time. Evaluation is performed by deterministic rollouts of the learned policies through the held-out validation and test Parquet splits in our `OfflineTradingEnv` (see Methods).

4 Methods

4.1 Problem formulation

We consider a finite-horizon decision process induced by a multivariate time series of market features at each trading day t, and an action that sets portfolio exposure to BTC. Actions are discrete and map to directional exposure (short, flat, long). The agent should improve risk-adjusted performance relative to sensible baselines (notably buy-and-hold BTC) on held-out periods under realistic frictions. Offline RL lacks access to the true environment dynamics at training time. Thus, the dataset may come from suboptimal behavior policies, inducing distributional shift between training data and the learned policy, which motivates conservative value learning.

4.2 Portfolio simulator and reward

Portfolio value evolves as

$$V_{t+1} = V_t\big(1 - c|\Delta w_t|\big)\big(1 + w_t(P_{t+1}/P_t - 1)\big),$$

with $w_t \in \{-1, 0, +1\}$ and $c =$ fee_rate + slippage_rate (default 5 bps fee, zero slippage in our experiments unless noted). The step reward is $\log(V_{t+1}/V_t)$.

4.3 Offline trading environment

A Gymnasium-compatible `OfflineTradingEnv` iterates sequentially over Parquet splits. Observations concatenate z-scored features with the current position (18 dimensions in our setup). The action space is discrete with values $\{-1, 0, +1\}$ (mapped to $\{0, 1, 2\}$ inside `d3rlpy`). Each step returns the next state, log-return reward, termination flag, and diagnostics (equity, drawdown, turnover, cost).

4.4 Buy-and-hold baseline

Buy-and-hold is a strong baseline because it captures the market's natural upward drift with zero decision-making overhead. Any learned agent must therefore overcome both transaction costs and the opportunity cost of deviating from a trend that historically rewards passive exposure. Finally, because it involves no parameter tuning and no exploitation of the state representation, any excess return over buy-and-hold can be attributed cleanly to learned skill rather than market beta.

4.5 Double DQN baseline

We train a Double DQN model using `d3rlpy` with an MLP encoder, ReLU activations, layer normalization, dropout, and a multi-critic ensemble. As a value-based baseline, it tests whether standard off-policy Q-learning without offline reinforcement learning corrections can learn a profitable trading policy from a fixed dataset, serving as a reference for CQL with conservative regularization and DistCQL with distributional extensions.

Training uses early stopping on a held-out validation split, selecting checkpoints by validation Sharpe ratio. The best model is reported and compared against CQL. The test set is used only for final evaluation and never for model selection.

4.6 Discrete Conservative Q-Learning (CQL)

Standard Q-learning trained on a fixed dataset tends to overestimate the value of rare or unseen actions, since the Bellman backup can extrapolate to out-of-distribution actions without corrective feedback from the environment. Conservative Q-Learning (CQL) mitigates this by introducing a regularizer that suppresses Q-values for actions favored by the policy but not supported by the dataset, while preserving in-distribution estimates (4). This produces a lower bound on the true value function, which is desirable when evaluating policies on held-out market data that cannot be resampled.

We use the discrete CQL implementation from `d3rlpy` with the same MLP backbone as DQN, along with weight decay, gradient clipping, a three-network Q ensemble, and early stopping based on validation Sharpe ratio (patience 4). Hyperparameters for DQN, CQL, and DistCQL are summarized in Table 2.

4.7 Distributional Conservative Q-learning (DistCQL)

DistCQL extends the discrete CQL framework by modeling return distributions using 51 quantiles per action. It introduces a tail-specific CQL penalty, weighted by α_{tail}, applied to the upper quantiles of under-supported actions, alongside a three-network critic ensemble with pessimistic aggregation (i.e., taking the minimum across critics). The standard CQL weight α and tail weight $\alpha_{\text{tail}} = \alpha/2$ are tuned jointly.

Intuitively, the mean CQL term penalizes overestimated Q-values for rarely observed actions, while the tail penalty further suppresses overly optimistic upper quantiles for out-of-distribution actions. This is particularly important when asymmetries in returns are only visible at the distributional level rather than in the mean. Model selection is based on validation Sharpe ratio, which is used for both checkpointing and early stopping.

At $\alpha = 1.0$, the conservative penalty is too strong, effectively suppressing all actions except the dominant "long" position (reflecting the bull-heavy training data), and collapsing the policy to a buy-and-hold strategy. To address this, we sweep $\alpha \in \{1.0, 0.5, 0.3\}$. We find $\alpha = 0.5$ (with $\alpha_{\text{tail}} = 0.25$) to be optimal, as it enables meaningful short exposure (12% of test days) while maintaining sufficient conservatism to prevent overfitting. In contrast, $\alpha = 0.3$ yields strong validation performance but poor test generalization (see Section 5.4).

Table 2: Hyperparameter comparison across all three algorithms.

Hyperparameter	DQN	CQL	DistCQL
Implementation	d3rlpy	d3rlpy	Custom PyTorch
CQL α	—	1.0	0.5
α_{tail}	—	—	0.25
Hidden layers	$[128, 128]$	$[256, 256]$	$[256, 256]$
Activation	ReLU	ReLU	ReLU
Layer norm / Dropout	Yes / 0.1	Yes / 0.1	Yes / 0.1
N critics	3	3	3
Learning rate	3×10^{-4}	1×10^{-4}	1×10^{-4}
Weight decay	10^{-4}	10^{-4}	10^{-4}
Grad clip (max norm)	1.0	1.0	1.0
Batch size	256	256	256
Discount γ	0.95	0.95	0.95
Target update interval	1 000	2 000	2 000
Total training steps	20 000	30 000	30 000
Steps per epoch	2 000	2 000	2 000
N quantiles	—	—	51 (tail-focused)
Action selection	argmax Q	argmax Q	pessimistic mean

Hyperparameter rationale. We set γ=0.95 because daily trading emphasizes medium-term rewards more than a near-1 discount typical of very long horizons. Batch size 256, weight decay 10^{-4}, and gradient clipping (norm 1) follow common stable deep-RL practice. DQN uses a higher learning rate (3×10^{-4}), fewer total steps (20,000), and faster target updates (1,000) because it has no conservative loss; CQL and DistCQL use 1×10^{-4}, 30,000 steps, and 2,000-step targets so pessimistic objectives can converge without unstable targets. We use smaller MLPs for DQN and $[256, 256]$ for CQL/DistCQL to give the latter enough capacity for pessimistic ensembles and 51 quantiles. CQL uses α=1 as in standard discrete CQL; DistCQL uses α=0.5 and α_{tail}=$\alpha/2$ from the sweep in Section 5.4. Three critics, dropout 0.1, and pessimistic aggregation (DistCQL) add diversity and robustness.

4.8 Ensemble via position averaging

We construct a two-model ensemble by combining DistCQL ($\alpha = 0.5$) and CQL. Each day, we average their real-valued position signals and round to the nearest valid action $\{-1, 0, +1\}$. This induces three regimes: if both models predict long, the ensemble goes long; if one predicts long and the other short, the average yields a neutral (flat) position; if both predict short, the ensemble goes short.

This consensus mechanism requires agreement between independently trained models before taking a directional position. As a result, it filters out spurious short signals from CQL, converting them into flat positions, while preserving high-confidence bearish signals when both models align. The ensemble operates entirely at inference time and requires no retraining.

4.9 Evaluation protocol

We roll out each learned policy on validation and test Parquet files and compare to buy-and-hold using the same cost model. We report total return, Sharpe ratio, maximum drawdown, cumulative position changes (turnover proxy), and action fractions (short, flat, long). Each model is trained with five random seeds (42, 1, 2, 3, 4); we report both single-seed and cross-seed summary statistics. Statistical significance tests follow Section 5.8.

4.10 Alternatives considered

We focus on offline value-based methods with discrete actions rather than online algorithms. Imitation learning and pure behavioral cloning are not the main comparison because we target value extrapolation and conservatism. Online fine-tuning is left out for the same reason as in standard offline RL benchmarks.

5 Experiments

5.1 Research questions

- How does regime shift between training data (bull-heavy) and validation or test splits affect offline Q-learning?
- Does regularized DQN still exhibit extrapolation error and policy instability across splits?
- Does mean-only CQL reduce that failure mode, and what is the cost in turnover and test performance versus buy-and-hold?
- How sensitive is DistCQL to the conservatism weight α, and can tuning it unlock active distributional trading?
- Can ensembling two complementary agents (DistCQL + CQL) beat buy-and-hold on both return and risk metrics?
- How robust are these findings across random seeds, and which differences are statistically significant?

5.2 Double DQN: extrapolation and turnover

Standard offline Q-learning can compound bootstrap error at OOD state-action pairs. Our Double DQN includes standard statistical regularization (dropout, layer normalization, weight decay, and a three-critic ensemble; see Table 2), yet lacks a conservative penalty on out-of-distribution actions. It fits the validation split extremely well but fails entirely on test.

Table 3 summarizes this failure mode: the learned policy achieves strong validation metrics but does not generalize to the held-out test regime.

Table 3: Double DQN results. Turnover counts cumulative position changes over the split.

Metric	DQN
Validation return	$+524\%$
Validation Sharpe	1.47
Validation turnover	461
Test return	-23.0%
Test Sharpe	-0.24
Test turnover	449

DQN has no mechanism to down-weight Q-values for actions that the batch barely supports. On validation (a bear regime) it assigns high values to shorting; turnover is large as the policy chases noisy advantages. On test (a recovery bull, Table 1) the same mechanism points the wrong way: about 53% of days are short during a period when passive long earns $+98\%$ cumulatively and achieves Sharpe 0.59. This matches the project thesis: extrapolation error, optimistic Q-values on rare actions, then greedy exploitation of those errors.

5.3 Mean CQL ($\alpha = 1$): conservative but still active

Discrete CQL uses the same data and evaluator. The best validation Sharpe occurs at epoch 12 (early stopping with patience four). Validation return is well below DQN, consistent with the CQL penalty preventing the "free lunch" from overstated Q-values.

Table 4: CQL results (seed 42, best checkpoint at epoch 12).

Metric	CQL
Validation return	$+129\%$
Validation Sharpe	0.64
Validation actions (S/F/L)	42% / 2% / 56%
Test return	$+92\%$
Test Sharpe	0.57
Test actions (S/F/L)	15% / 2% / 83%

On validation, CQL outperforms buy-and-hold in both Sharpe and return (0.64 vs. -0.06; $+129\%$ vs. -7.4%), as conservative Q estimates reduce overestimation while still allowing short positions (42% short, 56% long). On test, the policy is predominantly long (83% long, 15% short) and roughly matches buy-and-hold on Sharpe (0.57 vs. 0.59) while slightly trailing on return ($+92\%$ vs. $+97.9\%$).

During training, the short fraction increases from 2% at epoch 1 to 42% at epoch 12, while mean predicted Q remains negative and the conservatism proxy declines, which is consistent with pessimistic value estimates and increasingly sharp per-state action preferences.

5.4 DistCQL alpha tuning

The original DistCQL (α=1.0, α_{tail}=0.5) collapses to 100% long on test which is identical to buy-and-hold. The conservative penalty was strong enough to suppress all non-long actions because long is the most-represented action in the bull-heavy training data. Table 5 compares three conservatism levels.

Table 5: DistCQL alpha sweep (individual models). Actions are test-set fractions (short/flat/long).

α	α_{tail}	Best Epoch	Test Return	Test Sharpe	Test Max DD	Actions (S/F/L)
1.0	0.5	4	$+97.8\%$	0.59	-32%	0%/0%/100%
0.5	0.25	3	$\mathbf{+161\%}$	$\mathbf{0.84}$	-31%	12%/0%/88%
0.3	0.15	12	$+34.8\%$	0.26	-44%	25%/0%/75%

At α=0.5 the halved penalty allows the model to learn that shorting is sometimes justified while still anchoring to the data distribution. The distributional structure (51 quantiles) helps judge when shorting is warranted by examining the full return distribution rather than a point estimate. At α=0.3 the penalty is too weak: the model earns $+145\%$ on the bear-market validation split but shorts 25% of the time on the bull-market test, which is classic overfitting.

5.5 Ensemble: DistCQL (α=0.5) + CQL

Averaging positions from DistCQL (α=0.5) and CQL yields the best performance across all experiments (Table 6). Over 730 test trading days, the ensemble is long 75%, short 18%, and flat 7%, achieving a $+238\%$ return, Sharpe 1.09, and maximum drawdown of -22%.

Where the return comes from. On the 132 days when the ensemble takes short positions, the market declines on 55% of those days (72/132). Each correct short generates a double benefit relative to buy-and-hold: the ensemble gains while buy-and-hold loses. These short positions contribute $+20.8\%$ in log-return, corresponding to a total log-return advantage of $+0.415$ over buy-and-hold, approximately a 50% return gain from shorts alone. Combined with capturing 75% of the bull market through long positions, the total return reaches $+238\%$.

Why the ensemble beats either model alone. When evaluated in isolation, both CQL and DistCQL (α=0.5) produce short signals that include false positives. Position averaging acts as a consensus filter: when the models disagree (one long, one short), the ensemble goes flat rather than taking a risky directional bet. This converts spurious short signals into neutral positions while preserving high-conviction shorts where both models align. Because the ensemble's actions alter the trajectory each model observes (the current position is part of the state), the joint dynamics differ from either model's standalone behavior, concentrating short exposure on the strongest shared signals.

Three-model ensemble. Adding DQN as a third model degrades performance: any ensemble including DQN produces negative or near-flat test returns (e.g., $+11\%$ when all three are combined), as DQN's overfit shorting signal contaminates the average.

5.6 Overall comparison (single seed)

Table 6 consolidates held-out performance for the original seed (42). DQN overfits to validation ($+524\%$, Sharpe 1.47) then collapses on test (-23%). CQL stabilizes behavior but false-positive shorts cost it versus buy-and-hold. DistCQL shorts more selectively (12% vs. 15%), earning $+161\%$. The ensemble further benefits from consensus gating, reaching $+238\%$ with the lowest drawdown.

Table 6: Held-out metrics, seed 42. DistCQL uses α=0.5; ensemble averages DistCQL and CQL positions.

	Validation		Test			
Method	Sharpe	Return	Sharpe	Return	Max DD	Actions (S/F/L)
Buy-and-hold	-0.06	-7.4%	0.59	$+97.9\%$	-32%	0/0/100%
Reg. DQN	1.47	$+524\%$	-0.24	-23%	-62%	53/11/36%
CQL (α=1)	0.64	$+129\%$	0.57	$+92\%$	-37%	15/2/83%
DistCQL (α=0.5)	0.15	$+22\%$	0.84	$+161\%$	-31%	12/0/88%
Ensemble	0.46	$+78\%$	$\mathbf{1.09}$	$\mathbf{+238\%}$	$\mathbf{-22\%}$	18/7/75%

5.7 Seed robustness

Following Henderson et al. (7), we retrain each architecture with five random seeds (42, 1, 2, 3, 4) and evaluate every DistCQL$\times$CQL ensemble combination (25 pairs). Table 7 summarizes test-set performance across seeds.

Table 7: Test-set performance across 5 random seeds. Ensemble statistics are over all 25 DistCQL$\times$CQL pairs.

	DQN	CQL	DistCQL (α=0.5)	Ensemble (25 pairs)
Mean test return	$+24\%$	$\mathbf{+162\%}$	$+95\%$	$+154\%$
Median test return	-20%	$+156\%$	$+98\%$	$+166\%$
Seeds positive (/5)	2	5	4	24/25 (96%)
Seeds $>$ B&H (/5)	0	**4**	2	19/25 (76%)
Best seed return	$+97.7\%$	$+233\%$	$+168\%$	$+288\%$
Worst seed return	-27%	$+92\%$	-22%	-26%

CQL is the most seed-robust architecture. All five CQL seeds produce positive returns, four outperform buy-and-hold, and three converge to qualitatively similar strategies ($\sim$ 30–35% short, $\sim$ 60–65% long). The CQL penalty creates a basin of attraction in parameter space to which most initializations converge.

DistCQL is high-variance. Its loss landscape has two strong attractors—all-long and all-short—with a narrow ridge in between. When initialization lands on the ridge (seeds 42, 1), DistCQL produces the best individual returns ($+161$–168%). Two seeds instead collapse to extreme shorting (71–90% short) and underperform. The lower α=0.5 trades regularization strength for the ability to learn active trading.

DQN consistently overfits. No DQN seed beats buy-and-hold. Seeds with the highest validation returns show the worst test returns—a textbook signature of extrapolation error.

The ensemble moderates variance. Among all 25 DistCQL$\times$CQL pairs, 96% produce positive returns and 76% beat buy-and-hold. The original seed-42 ensemble ($+238\%$) is above the grid mean ($+154\%$) but is not the maximum ($+288\%$, from DistCQL seed 2 + CQL seed 3).

5.8 Statistical significance

We apply t-tests to individual models and bootstrap resampling to the ensemble grid (10,000 resamples), following the recommendations of Henderson et al. (7).

Comparison to buy-and-hold. The DistCQL+CQL ensemble significantly outperforms buy-and-hold (one-sample t-test, t=3.52, p=0.002): across all 25 seed pairs, the mean return is $+154\%$ vs. $+98\%$. CQL individually is marginally significant (p=0.074), suggesting significance would be reached with additional seeds. DQN is marginally worse than buy-and-hold (p=0.064).

Pairwise comparisons. Using two-sample t-tests across shared seeds, CQL significantly outperforms DQN on both test returns (t=3.50, p=0.008) and Sharpe ratio (t=2.42, p=0.042). CQL versus DistCQL (p=0.16) and DistCQL versus DQN (p=0.17) are not significant. A bootstrap power analysis estimates 95% power for CQL versus DQN but only $\sim$27% for CQL versus DistCQL, suggesting 15–20 seeds per model would be needed for 80% power on that comparison.

6 Conclusion

Training on a bull-heavy offline batch and evaluating on bear and bull splits makes extrapolation error visible. Regularized DQN validates the concern: high validation Sharpe and turnover, then test failure across all five seeds (no seed beats buy-and-hold). Mean CQL is the most seed-robust architecture: all five seeds produce positive returns and four beat buy-and-hold. Tuning DistCQL's conservatism weight to α=0.5 unlocks active distributional trading in favorable initializations, but the model is more seed-sensitive, with two of five seeds collapsing to degenerate strategies. The central statistical result is that the DistCQL+CQL ensemble significantly outperforms buy-and-hold (one-sample t-test, $p = 0.002$), with a mean return of $+154\%$ versus $+98\%$ across all 25 seed pairs. The seed-42 ensemble achieves a $+238\%$ test return, with a Sharpe ratio of 1.09 and a maximum drawdown of -22%, outperforming buy-and-hold by $2.4\times$. The best pair in the grid (DistCQL seed 2 + CQL seed 3) reaches $+288\%$. Two-sample t-tests show that CQL outperforms DQN on both test returns ($p = 0.008$) and Sharpe ratio ($p = 0.042$), and a bootstrap power analysis estimates approximately 95% power for the CQL vs. DQN return comparison, though individually, CQL remains only marginally significant relative to buy-and-hold ($p = 0.074$). The progression from DQN (extrapolation) to CQL (conservatism) to tuned DistCQL (distributional awareness) to the ensemble (consensus gating) illustrates how layering offline RL techniques can move from failure to stronger out-of-sample performance, with the ensemble providing the clearest evidence against buy-and-hold under our bootstrap protocol. The seed analysis reveals that regularization strength is the key factor determining robustness. While limitations include a single asset, daily frequency, and simplified costs, where multi-asset and intraday extensions would stress the same offline issues under richer microstructure, next steps include risk-sensitive action selection (e.g., CVaR-based policies) using the learned quantiles and extending to multi-asset portfolios.

References

[1] V. Mnih et al., "Human-level control through deep reinforcement learning," Nature, vol. 518, pp. 529-533, 2015.

[2] G. Lucarelli and M. Borrotti, “A deep Q-learning portfolio management framework for the cryptocurrency market,” Neural Comput. Appl., vol. 32, pp. 17229-17244, 2020.

[3] S. Levine, A. Kumar, G. Tucker, and J. Fu, “Offline reinforcement learning: Tutorial, review, and perspectives on open problems,” arXiv preprint arXiv:2005.01643, 2020.

[4] A. Kumar, A. Zhou, G. Tucker, and S. Levine, “Conservative Q-learning for offline reinforcement learning,” in Proc. NeurIPS, 2020.

[5] M. G. Bellemare, W. Dabney, and R. Munos, “A distributional perspective on reinforcement learning,” in Proc. ICML, 2017.

[6] W. Dabney et al., “Implicit quantile networks for distributional reinforcement learning,” in Proc. ICML, 2018.

[7] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, “Deep reinforcement learning that matters,” in Proc. AAAI, 2018.